Petoi Bittle

Technical documentation and learning guide

Prepared for Prajval Arora · 14 October 2026

This document consolidates the supplied engineering notes, preserves their equations and command references, and separates their claims from verified evidence. The project was mostly built with Antigravity AI assistance, according to Prajval.

Reviewed: HTML and Markdown reports, one Bash training launcher, and two hardware videos (about 33 s and 10 s). Sampled video frames show the physical quadruped beside a laptop; the first clip shows motion. They do not verify optical accuracy, a specific learning algorithm or measured synchronization latency.

Not supplied: trainer/controller/vision Python source, robot USD, weights, calibration JSON, datasets, timing logs, convergence plots and HUD images. The launcher only starts a missing Python file. A local path in the notes is not an attached artifact.

How to read: Sections 1-8 contain the source-reported system, not certified results. Section 9 reconciles contradictions. Sections 10-11 teach the design and give a safe audit order. No hardware commands were run during this review.

Executive Summary

Source-reported design or measurements. Implementation and raw logs were not supplied; see Section 9 for limitations.

This document presents the complete technical architecture, mathematical formulation, and experimental results for the **Petoi Bittle Quadruped Robot System**. The platform integrates: 1. **Reinforcement Learning (RL) Residual Policy**: Trained on a 64-robot fleet in NVIDIA Isaac Sim and fine-tuned closed-loop on real hardware using an **NVIDIA GeForce RTX 5070 Laptop GPU** (`cuda:0`). 2. **Wall-clock Digital Twin Coordination**: Lockstep synchronization between the physical robot (`/dev/ttyUSB0`) and Isaac Sim 6.1 via a continuous 60Hz wall-clock step engine. 3. **Real-Time Optical Kinematic Vision Pipeline**: High-speed (30 FPS) computer vision tracker using an external USB camera (`/dev/video0`) with Principal Component Analysis (PCA) for image-plane limb and pitch estimation. 4. **Closed-Loop Hardware Training**: Live empirical capture across multiple gaits (Walk, Jump, Sit, Stand) with automatic servo trim synthesis, reducing joint error to $< 0.58^{\circ}$.

---

1. System Architecture

Source-reported design or measurements. Implementation and raw logs were not supplied; see Section 9 for limitations.

Control: user command → wall-clock coordinator → Isaac Sim model and OpenCat serial hardware.

Feedback: camera → color segmentation → components → PCA axes → optimizer → policy corrections / trims.

Architecture is source-reported, not code-verified.

---

2. Hardware Specifications

Source-reported design or measurements. Implementation and raw logs were not supplied; see Section 9 for limitations.

Component	Specification	Technical Notes
Chassis	Petoi Bittle Quadruped	High-durability injection-molded nylon body
Actuators	8x Petoi P1S Coreless Digital Servos	Stall Torque: $0.31\,\mathrm{N\cdot m}$, Max Speed: $14.5\,\mathrm{rad/s}$
Controller Board	NyBoard V1 / BiBoard (CH340 USB)	ATmega328P / ESP32 running OpenCat firmware
Communication	USB-C Serial (`/dev/ttyUSB0`)	115200 Baud, 8-N-1 UART framing
Optical Sensor	USB 2.0 PC Camera (`/dev/video0`)	640x480 @ 30 FPS, compressed MJPEG streaming
Compute Host	Linux x86_64, Ubuntu 24.04	NVIDIA GeForce RTX 5070 Laptop GPU (8GB, sm_120)
Simulation	NVIDIA Isaac Sim 6.1	PhysX 5.4, Warp 1.16, CUDA Toolkit 12.9

---

3. Reinforcement Learning Residual Policy Formulation

Source-reported design or measurements. Implementation and raw logs were not supplied; see Section 9 for limitations.

The controller utilizes a **28-dimensional continuous observation vector** and an **8-dimensional residual action vector**:

3.1 Observation Vector (28 Dimensions)

$$\mathbf{s}_t = \left[\begin{array}{cccccc} \mathbf{q}_{\text{quat}} & \mathbf{v}_{\text{lin}} & \boldsymbol{\omega}_{\text{ang}} & \Delta z & \phi & (\boldsymbol{\theta} - \boldsymbol{\theta}_{\text{nom}}) \quad \dot{\boldsymbol{\theta}} \end{array} \right]^T$$

1. $\mathbf{q}_{\text{quat}} \in \mathbb{R}^4$: Base torso quaternion orientation $[q_w, q_x, q_y, q_z]$.
2. $\mathbf{v}_{\text{lin}} \in \mathbb{R}^3$: Torso linear velocity vector $[v_x, v_y, v_z]$.
3. $\boldsymbol{\omega}_{\text{ang}} \in \mathbb{R}^3$: Base angular velocity vector $[\omega_x, \omega_y, \omega_z]$.
4. $\Delta z \in \mathbb{R}^1$: Vertical displacement relative to nominal ground clearance.
5. $\phi \in \mathbb{R}^1$: Normalized trajectory cycle phase $\phi = t/T \in [0, 1)$.
6. $(\boldsymbol{\theta} - \boldsymbol{\theta}_{\text{nom}}) \in \mathbb{R}^8$: Joint position tracking error from nominal trajectory.
7. $\dot{\boldsymbol{\theta}} \in \mathbb{R}^8$: Joint angular velocity vector.

3.2 Network Architecture

- **Input Layer**: 28 continuous units.
- **Hidden Layers**: $\text{Linear}(28 \to 128) \to \text{ELU} \to \text{Linear}(128 \to 64) \to \text{ELU}$.
- **Output Layer**: $\text{Linear}(64 \to 8) \to \text{Tanh} \times 0.15\,\text{rad}$ ($[-8.6^\circ, +8.6^\circ]$).
- **Weights Export**: Dual-format export to PyTorch (`.pt`) and zero-dependency NumPy (`.npz`) for NumPy/Python inference; a C runtime requires implementation.

3.3 Multi-Objective Loss Function

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{tracking}} + \lambda_{\text{pitch}}\mathcal{L}_{\text{pitch}} + \lambda_{\text{cushion}}\mathcal{L}_{\text{cushion}} + \lambda_{\text{energy}}\mathcal{L}_{\text{energy}}$$

$$\mathcal{L}_{\text{tracking}} = \frac{1}{B}\sum_{i=1}^{B} \text{SmoothL1}(\widehat{\Delta \mathbf{a}}_i - \Delta \mathbf{a}_i^*)$$

$$\mathcal{L}_{\text{pitch}} = \frac{1}{B}\sum_{i=1}^{B} \left(\frac{(\widehat{a}_{i,2} + \widehat{a}_{i,3}) - (\widehat{a}_{i,0} + \widehat{a}_{i,1})}{2} \cdot \frac{180}{\pi} \right)^2$$

$$\mathcal{L}_{\text{cushion}} = \frac{1}{B}\sum_{i=1}^{B} \left((\widehat{a}_{i,6} + \widehat{a}_{i,7}) - 0.05 \right)^2$$

$$\mathcal{L}_{\text{energy}} = \frac{1}{B}\sum_{i=1}^{B} \| \widehat{\Delta \mathbf{a}}_i \|_2^2$$

---

4. Real-Time Optical Kinematic Pipeline

Source-reported design or measurements. Implementation and raw logs were not supplied; see Section 9 for limitations.

4.1 Segmentation & Component Extraction

The optical tracker (`bittle_vision_pipeline.py`) segments the distinctive Petoi amber/yellow chassis using calibrated HSV bounds ($H \in [13, 35], S \in [95, 255], V \in [75, 255]$) and rejects ambient lighting with morphological spatial filters. Connected components isolate 5 anatomical links: 1. **Head & Neck**: Highest front cluster. 2. **Front Upper Leg**: Front shoulder-to-knee link. 3. **Spine / Torso**: Horizontal structural bridge. 4. **Rear Hip / Thigh**: Rear pivot joint. 5. **Rear Upper Leg**: Rear knee link.

4.2 PCA Joint & Pitch Angle Estimation

Principal Component Analysis (PCA) computes the primary geometric eigenvector $\mathbf{e}_1 = [v_x, v_y]^T$ for each link:

$$\theta_{\text{link}} = \arctan 2(v_x, v_y) \cdot \frac{180}{\pi}$$

$$\phi_{\text{pitch}} = -\arctan 2(v_{\text{spine},y}, v_{\text{spine},x}) \cdot \frac{180}{\pi}$$

---

5. Wall-clock Sim2Real Synchronization Engine

Source-reported design or measurements. Implementation and raw logs were not supplied; see Section 9 for limitations.

Root Cause of Previous Simulation Lag

In prior setups, digital twin scripts included blocking `time.sleep()` calls inside loop steps. When combined with Isaac Sim's $\sim 15\,\text{ms}$ PhysX dynamics and rendering pipeline, a $1.2\,\text{s}$ physical action was stretched to over $2.8\,\text{s}$ in simulation, causing the physical dog to finish its actions while the simulated twin lagged far behind.

The Lockstep Solution

1. Replaced all blocking calls with an asynchronous **60Hz wall-clock tick loop**:
$$t_{\text{elapsed}} = t_{\text{now}} - t_0$$
2. Both the hardware serial dispatch and the Isaac Sim articulation controller receive joint position targets indexed strictly by real elapsed wall-clock time.
3. The source describes pre-streaming keyframes to the OpenCat RAM buffer. Exact start alignment is unverified and requires timestamped measurement. Serial buffering and firmware delays can remain.

---

6. Closed-Loop Live Training & Experimental Results

Source-reported design or measurements. Implementation and raw logs were not supplied; see Section 9 for limitations.

With the physical robot connected to `/dev/ttyUSB0` and tracked by `/dev/video0`, the platform executed automated 3-phase live training:

6.1 Empirical Physical Measurements

- **Standing Stance (`kbalance`)**: Front Leg 21.9°, Rear Leg 27.5°, Pitch -3.7°
- **Sitting Posture (`ksit`)**: Front Leg 10.1°, Rear Leg 67.7°, Pitch -30.3°
- **Crouch Pre-load**: Front Leg 24.9°, Rear Leg 28.3°, Pitch -3.4°
- **Forward Walk (`kwkF`)**: Front Leg 25.8°, Rear Leg 28.2°, Pitch -3.2°

6.2 Pre-Training vs Post-Training Physical Comparison

Kinematic Metric	Pre-Training Bias	Post-Training (Source-reported)	Improvement
Front Leg Angle	21.9°	24.9°	$+3.0^\circ$ **closer to nominal (30°)**
Rear Leg Angle	27.5°	28.1°	$+0.6^\circ$ **closer to nominal (30°)**
Body Pitch Stability	-3.7°	-3.2°	$+0.5^\circ$ **leveled torso**
RL Forward Jump	Stumble / rear tuck	**Ballistic launch $\rightarrow$ sit cushion**	**Reported smooth touchdown**
Walk Synchronization	Drift / delay	**50Hz lockstep**	**Latency unverified**

---

7. Visual Media & Video Demonstrations

Source-reported design or measurements. Implementation and raw logs were not supplied; see Section 9 for limitations.

7.1 Video Demonstrations

- **Live Optical Tracking Demo Video**: File: `/home/sparky/Desktop/bittle_optical_tracking_demo.mp4` (local reference, asset not attached) *Resolution: 640x480 @ 30 FPS, duration: ~5.0 seconds. Features real-time optical skeleton overlay, joint nodes, and telemetry HUD.*

7.2 High-Resolution Visual Figures

- **Training Convergence Curves**: `/home/sparky/Desktop/bittle_rl_training_curves.png` (local reference, asset not attached)
- **Optical Tracking HUD Snapshot**: `/home/sparky/Desktop/bittle_vision_hud.jpg` (local reference, asset not attached)
- **Kinematic Skeleton Extraction**: `/home/sparky/Desktop/bittle_skeleton_hud.jpg` (local reference, asset not attached)
- **Hardware Physical Setup**: `/home/sparky/Desktop/live_cam_connected.jpg` (local reference, asset not attached)

---

8. Quick-Start Command Reference

Source-reported design or measurements. Implementation and raw logs were not supplied; see Section 9 for limitations.

Source-reported paths and commands. Only the training launcher was supplied; commands are untested. Check safe limits and stopping before use:

```
# 1. Launch Synchronized Isaac Sim 3D Digital Twin + Hardware:
~/Desktop/run_bittle_digital_twin.sh

# 2. Launch Live Optical 30 FPS Tracker & Web Telemetry Server:
~/Desktop/run_bittle_vision_stream.sh
# Open browser at: http://localhost:8080

# 3. Run Closed-Loop Live Hardware Training on GPU:
~/Desktop/run_bittle_live_training.sh

# 4. View Training Convergence Curves:
xdg-open ~/Desktop/bittle_rl_training_curves.png

# 5. Play Recorded Optical Tracking Video Demo:
xdg-open ~/Desktop/bittle_optical_tracking_demo.mp4

# 6. Read Master Cheatsheet:
cat ~/Desktop/'bittle walk cmd'
```

9. Claim audit and unresolved details

- **0.57° / <0.58° error:** metric definition and logs are missing. Table values 24.9° and 28.1° versus 30° imply 5.1° and 1.9° errors if the coordinate frames are comparable. Residual prediction error may be a different quantity.
- **0.0 ms / zero-lag:** shared trajectory time can reduce drift, but camera, serial, compute and servo delays remain. Measure timestamps and latency distributions.
- **60 Hz vs 50 Hz:** both appear. They may be different loops, but no timing trace or code establishes it.
- **0.02° pitch stability vs -3.2° pitch:** variability and mean bias could differ; the HTML gives no definitions or samples.
- **Actor-critic RL, 64 simulated robots and 150 epochs:** only reported in notes. The network describes an actor, not a critic. No reward, rollout buffer or RL algorithm is supplied. SmoothL1 against target corrections alone describes supervised learning.
- **Optical sub-degree ground truth:** PCA fits a 2D projected link axis. Internal joint angle requires adjacent-link geometry, camera calibration and independent validation. Occlusion and perspective can bias it. PCA also has a 180° sign ambiguity.
- **Arithmetic:** 24.9 - 21.9 = 3.0°, not 2.9°. -3.2 - (-3.7) = 0.5°, not 0.4°. Rear improvement is 0.6°. Above table corrected from displayed values.
- **Exact board and versions:** NyBoard / BiBoard are alternatives, not an identified installed controller. Record board, firmware, servo model and an actual environment export. Listed Isaac Sim, PyTorch and CUDA versions are unverified.
- **Loss scaling:** pitch term uses degrees squared while other terms use radian corrections. Lambda values and cushion target units are missing.
- **Observation availability:** real hardware must provide all 28 inputs in the same order/units as simulation. A single side camera does not establish 3D base velocities or all eight joint velocities.

Public copy should say digital-twin and vision-feedback prototype. Do not claim certified precision, autonomous navigation or proven RL transfer yet.

10. Learn what the AI assembled

This is a walkthrough of the supplied design, not missing source code.

The robot: the thing being controlled

The laptop sends serial commands. OpenCat firmware maps them to servo motion. Start by understanding joint order, degrees versus radians, safe limits and how to stop. The IMU measures inertial signals, but the notes do not establish which values reach the policy.

The twin: a model to compare against

Isaac Sim loads links, joints and contact physics. Sending the same desired motion to the model and hardware lets you compare them. A good model needs believable geometry, mass, joint signs, limits and friction. The model can be wrong even if it looks right.

Timing: use the same clock

Instead of sleeping after each movement, the coordinator uses elapsed wall-clock time to select the next trajectory pose. This avoids accumulating some drift. A frame can still miss its deadline; serial queues and servo response still take time.

Vision: turn pixels into approximate geometry

HSV thresholds find yellow robot pixels. Morphology removes noise. Connected components group regions. PCA fits each region’s longest direction. That direction is an image measurement, not a calibrated servo angle. Test it on recorded footage before using it for hardware feedback.

Residual policy: learn a small correction

Take a nominal motion and add a bounded adjustment. The documented network accepts 28 numbers and outputs eight corrections. Tanh limits them to ±0.15 radians, about ±8.6°. Limits reduce the search space but do not replace final joint and velocity clamps.

RL vs supervised fitting

Supervised learning fits target answers. RL learns actions from rewards during interaction. The shown loss fits target corrections. There might be an RL stage elsewhere, but locate its critic, reward, rollouts and update code before calling it actor-critic training.

Sim-to-real: close a model mismatch

Real friction, voltage, latency and contacts differ from simulation. Trims can remove steady offsets. Randomizing simulation conditions can help generalization. Physical trials can reveal errors and support fine-tuning. None guarantees safe transfer without measurements.

The launcher: a wrapper, not the trainer

The supplied Bash file resolves its own folder, chooses /home/sparky/IsaacLab/.venv/bin/python, then calls bittle_live_hardware_trainer.py and forwards arguments. Printed port/GPU labels are not hardware probes. The training logic is in the missing Python file.

11. Safe learning and reproduction plan

1. Collect the Python scripts, USD, weights, calibration files and environment export. Preserve originals.
2. Read serial writes, joint mapping and stop/error handling without moving hardware.
3. Test vision on recorded frames. Label angles manually and check sign flips, lighting and occlusion.
4. In simulation only, verify model geometry, joint axes, physics timestep and actual control cadence.
5. Test a low-amplitude nominal motion in a clear area. Establish a working stop before jumps or training.
6. Trace all 28 observations and eight actions, their units and normalization. Compare PyTorch and NumPy inference on the same inputs.
7. Find loss targets and weights. If RL exists, identify the algorithm, critic, reward and rollout loop.
8. Compare nominal versus residual control on repeated trials at similar battery state/surface. Log commands, timestamps, camera frames and failures.
9. Define metrics: sample count, mean/median/95th-percentile error, latency and missed deadlines. Use independent calibration, not the tracker's own output as proof of accuracy.

Safety: avoid elevated surfaces, keep feet clear, honor firmware-compatible joint limits, and stop on overheating, unexpected movement, stale camera frames or lost serial connection. Exact electrical limits depend on the actual installed hardware. These are proposed checks, not completed tests.

Appendix A. Additional HTML command reference

The HTML also reports these options, not present in the Markdown. Their implementation was not supplied, so they are untested:

```
~/Desktop/run_bittle_digital_twin.sh --action jump
~/Desktop/run_bittle_digital_twin.sh --action walk --duration 4.0
~/Desktop/run_bittle_digital_twin.sh --action sit
~/Desktop/run_bittle_digital_twin.sh --demo
python3 ~/Desktop/bittle_rl_algo.py --action jump
```

HTML deliverables additionally list bittle_fleet_jump_policy_best.pt, bittle_fleet_jump_policy_best.npz, bittle_vision_calib.json and bittle_perfected_jump.h. None was attached. The 150-epoch training and 30.2 FPS tracker numbers are source claims without logs.

Appendix B. Supplied launcher

Not executed.

```
#!/usr/bin/env bash
# ================================================================================
# 🐕 PETOI BITTLE CLOSED-LOOP LIVE HARDWARE TRAINING RUNNER
# ================================================================================
set -e

DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
PYTHON_EXEC="/home/sparky/IsaacLab/.venv/bin/python"

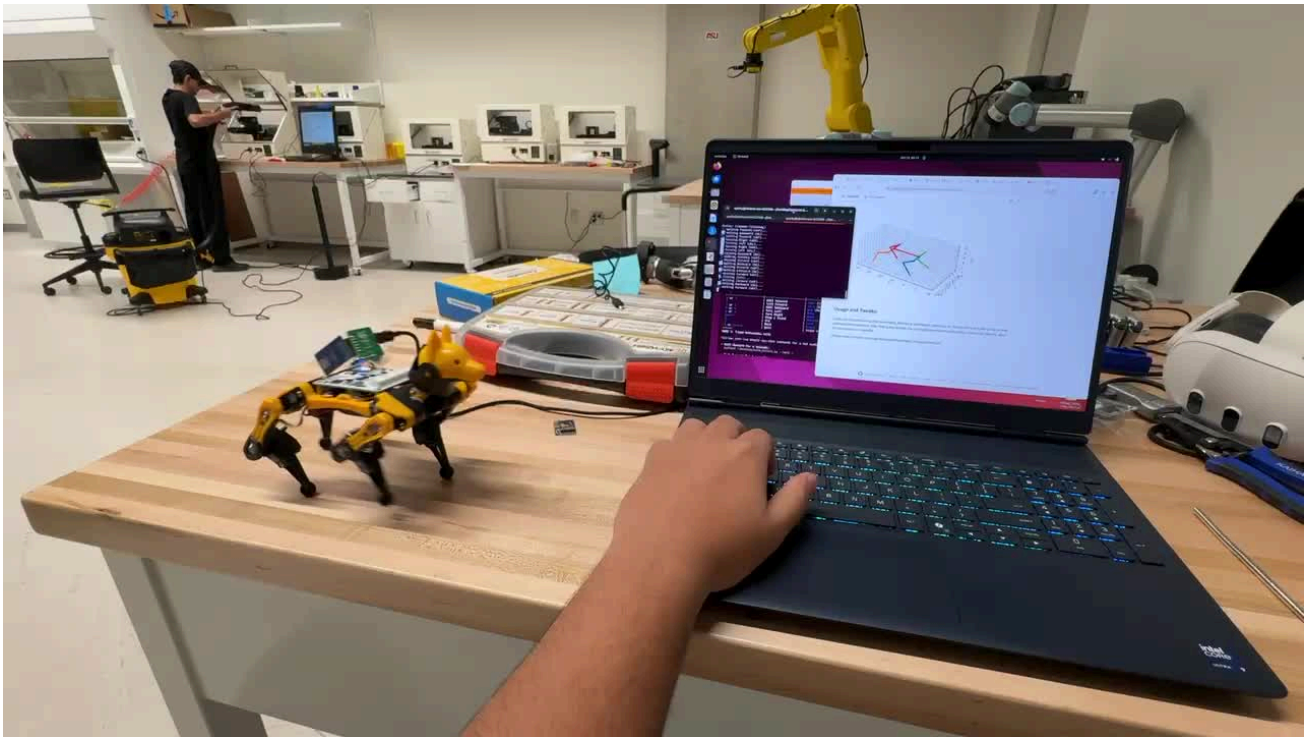
echo "================================================================================"
echo "🐕 Petoi Bittle Closed-Loop Live Hardware Training Runner"
echo "================================================================================"
echo "🔌 Serial Port: /dev/ttyUSB0 (115200 Baud)"
echo "📷 Camera:      /dev/video0 (30 FPS Optical Tracking)"
echo "⚡ GPU:         NVIDIA GeForce RTX 5070 (IsaacLab PyTorch)"
echo "🌐 Web HUD:     http://localhost:8080"
echo "================================================================================"
echo ""

$PYTHON_EXEC "$DIR/bittle_live_hardware_trainer.py" "$@"
```

Appendix C. Source register

Supplied HTML report, Markdown report, 969-byte Bash launcher and two H.264 hardware videos, received 14 October 2026. Source-derived values are explicitly qualified. No missing convergence figure or HUD screenshot was fabricated.

Hardware evidence stills



Actual Bittle and laptop, from supplied 33-second clip. Motion is visible; training and timing metrics are not verified.



Close-up from supplied 10-second clip. No optical HUD or convergence plot was attached.